Week 10 part 1

✅ Shared Memory

- A general scheme of using shared memory (Server)
- A general scheme of using shared memory (client)
- programming shared memory
  - ftok()
  - IPC_PRIVATE
  - Asking for a Shared Memory Segment - shmget()
  - Asking for a Shared Memory Segment - shmat()
  - Detaching a Shared Memory Segment - shmdt()
  - Removing a Shared Momoery Segment - shmctl()
- Parent and Child Share Memory
- Client-Server Shared Memory

> ## Content

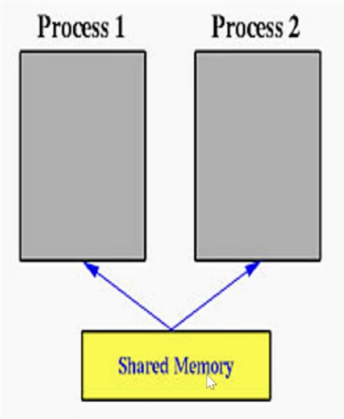
✅ Shared Memory

(the idea is 1 process ask for share memory → like that process said that i gonna allow some other process to use this share memory → the point is you have to do synchronization by yourself)

- when we talk about share memory, we have to talk about synchronization

What is Shared Memory?

- The figure shows two processes and their address spaces.
- The yellow rectangle is a shared memory attached to both address spaces and both process 1 and process 2 can have access to this shared memory as if the shared memory is part of its own address space.
- Note that, race conditions may occur if memory accesses are not handled properly.



What is Shared Memory? (Contd.)

- Shared memory is a feature supported by UNIX System V, including Linux, SunOS and Solaris.
- One process must explicitly ask for an area, using a key, to be shared by other processes. This process will be called the server.
- All other processes, the clients, that know the shared area can access it.
- However, there is no protection to a shared memory and any process that knows it can access it freely.
- To protect a shared memory from being accessed at the same time by several processes, a synchronization protocol must be setup.

Let's divide program into 2 parts

- Server → process that create shared memory
- Client → process that used that shared memory

📍 A general scheme of using shared memory (Server)

- For a Server, it should be started before any client. The server should perform the following tasks:

  🌷 Shmget()

  - Ask for a shared memory with memory key and memorize the returned shared memory ID. This is performed by system call shmget()
    - is used on server side → create share memory

- return shared memory id (must be unique)
    - (how?)it depend on the key that you pass to shmget()
    - the client process that want to use the shared memory must use the same key as server, so it can get the same share memory id.

🌷 **Shmat()**

- Attach this shared memory to the server's address space with system call shmat()
    - when you want to access memory from computer → in normal programming, you use variable (you declare variable because you wnat to refer to memory, but don't want to remember the memory address)
    - memory address will be reserved when you use shmget(), and when you want to use the share memory → must use variable to represent the share memory **and to link variable to memory use shmat()**
- Initialize the shared memory, if necessary
    - if the server want to initialize memory, it can do
        - because now we have variable attach to memory, share memory can be use like memory
- Server program can wait for client to use share memory
- Do something and wait for all clients' completion

🌷 **Shmdt()**

(when all client already use share memory, noone gonna use it anymore)

- Detach the shared memory with system call shmdt()

🌷 **Shmctl()**

(delete share memory) (actually it can be use to do lot of thing other than delete)

- Remove the shared memory with system call shmctl()

📍 A general scheme of using shared memory (Client)

- For the client part, the procedure is almost the same:

  🌷 **Shmget()**

  - Ask for a shared memory with the same memory key and memorize the returned shared memory ID
    - client use shmget()

  🌷 **Shmat()**

  - Attach this shared memory to the client's address space.
    - shmat to attach

  Use the memory

  🌷 **Shmdt()**

  - call shmdt
  - Detach all shared memory segments, if necessary.
  - Exit

- client used almost the same function as server, but client never shmctl() → delete the memory is the task of server

✅ Programming Shared Memory

Header file

- #include <sys/types.h>
- #include <sys/ipc.h>
- #include <sys/shm.h>

Key

- Unix requires a key of type key_t defined in file sys/types.h for requesting resources such as shared memory segments, message queues and semaphores.
- A key is simply an integer of type key_t; however, you should not use int or long, since the length of a key is system dependent.
- There are three different ways of using keys, namely:
  - a specific integer value (e.g., 123456)
  - a key generated with function ftok()
  - a uniquely generated key using IPC_PRIVATE (i.e., a private key).

- key_t is integer → it mean we can specify number like 123455 as a key, but we don't really specify a key like a 123456
  - people tend to use number that easy to rememner, and maybe more than 1 user of those machine use this number.
  - not reccomened → but for testing purpose → Ok → like if you run on you rmachine → you sure noone will create same key
- way to create a key
  - ftok(), IPC_PRIVATE

📍 ftok()

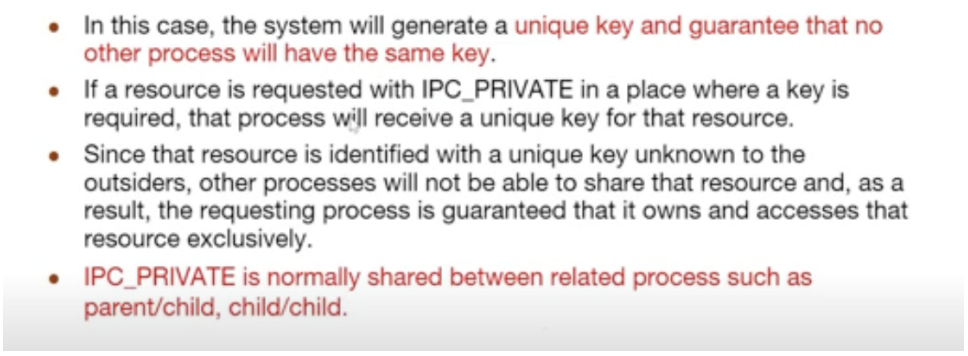
- The ftok() function has the following prototype:
- key_t ftok(const char *path, /* a path string */
- int id /* an integer value */);
- Function ftok() takes a character string that identifies a path and an integer (usually a character) value, and generates an integer of type key_t based on the first argument with the value of id in the most significant position.
- Thus, as long as processes use the same arguments to call ftok(), the returned key value will always be the same.
- The most commonly used value for the first argument is ".", the current directory.

- need 2 parameter
  1. first one is path, we always have to specify the path to a file
  2. id, any number then ftok will use a path and number to generate a key
- why this consider better than using integer number directly?
- way to normally use

- If all related processes are stored in the same directory, the following call to ftok() will generate the same key value:
  - #include <sys/types.h>
  - #include <sys/ipc.h>
  - key_t SomeKey;
  - SomeKey = ftok(".", 'x');

- . means current folder → in home folder should be unique.
- if someone want to access shared memory you create → can they do that
  - yes, they can specify first argument at your home folder. → they can access your shared memory
    - so can't guaranteed your key gonna be unique.
    - if you want to get 100% guarantee that your key is unique

📍 IPC_PRIVATE


- In this case, the system will generate a unique key and guarantee that no other process will have the same key.
- If a resource is requested with IPC_PRIVATE in a place where a key is required, that process will receive a unique key for that resource.
- Since that resource is identified with a unique key unknown to the outsiders, other processes will not be able to share that resource and, as a result, the requesting process is guaranteed that it owns and accesses that resource exclusively.
- IPC_PRIVATE is normally shared between related process such as parent/child, child/child.

- this IPC_PRIVATE can guaranteed that you rkey gonna be unique but there are one limitation
  - how can you send this key to some other process
    - this technique can be use by family of process (parent, child, sub child,.....)

📍 Asking for a Shared Memory Segment - shmget()

Asking for a Shared Memory Segment - shmget()

- It is defined as follows:
- shm_id = shmget(key_t k, /* the key for the segment */
- int size, /* the size of the segment */
- int flag); /* create/use flag */
- In the above definition, k is of type key_t or IPC_PRIVATE. size is the size of the requested shared memory.
- The purpose of flag is to specify the way that the shared memory will be used. For our purpose, only the following two values are important:
- IPC_CREAT | 0666 for a server (i.e., creating and granting read and write access to the server)
- 0666 for any client (i.e., granting read and write access to the client)
- If shmget() can successfully get the shared memory, its function value is a non-negative integer, the shared memory ID; otherwise, the function value is negative.

- 3 parameter
  1. key_t → key you can generate from the method above
  2. size → size of share memory you need to ask for (in bytes)
  3. flag
- 0666 → granting read and write to this share memory
  - why 0666 → in computer, if we start number with 0 → it mean we talking about octadecimal number
    - file permission → divided into 3 parts

```
-rw-r--r-- 1 sarun sarun  1836 Aug 24  2022 client.c
-rwxr-xr-x 1 sarun sarun 17088 Oct  6  2022 server
-rw-r--r-- 1 sarun sarun  2750 Oct  6  2022 server.c
-rw-r--r-- 1 sarun sarun  2936 Aug 24  2022 shm-01.c
-rw-r--r-- 1 sarun sarun   117 Aug 24  2022 shm-02.h
sarun@sarun-flowx13:~/os_course/sharemem$
```

      - first part → for owner
      - second part → user in the same group of the owner
      - last part → some other user in the system
      - r → read, w → write, x → execute

- rwx r— r— → 111 100 100 → 744 → (octadecimal) 0744
- 0666 → rw- rw- rw-

For server and client, they use the same function but client don't have to use IPC_CREATE

- shm_id → return from shmget()
  - if any process use the same key → they will get the same shared memory id (shm_id)

Example (using share memory id by the parent)

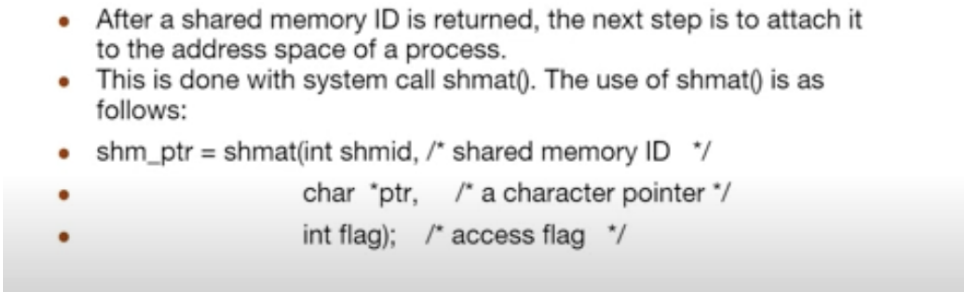
```
#include  <sys/types.h>
#include  <sys/ipc.h>
#include  <sys/shm.h>
#include  <stdio.h>
int  shm_id;    /* shared memory ID      */
 shm_id = shmget(IPC_PRIVATE, 4*sizeof(int), IPC_CREAT | 0666);
if (shm_id < 0) {
    printf("shmget error\n");
    exit(1);
}
```

If a client wants to use a shared memory created with IPC_PRIVATE, it must be a child process of the server, created after the parent has obtained the shared memory, so that the private key value can be passed to the child when it is created.

- parent create using IPC_PRIVATE
- 4 * size of int → if int require 4 bytes → you asking for 16 bytes of memory. you may ask → can we specify 16 here instead of 4*sizeof(int)
  - yes, but your program might not be portable, some other machine it might not use 4 bytes for integer.
- this memory area can be read and write by server(0666)

If for some reason, you can't get memory, shmget will return negative number back

📍 Asking for a Shared Memory Segment - shmat()



- 3 parameters → but for the last 2, we'll be use the default one
  - the first one shmid → you have to use shmid that you get from shmget()
    - and the variable must be pointer variable
    - and the pointer must be the same type of memory you ask for → (like you ask for memory for integer)
      - shm_ptr → must be pointer to integer
      - do you know why pointer in c must be the same type to what it point to?

```
int a[4];

int *p = a;

*(p+1)  === a[1]
```

        - if a start at 1000, (p+1) means 1004, not 1001 (add number of byte require by data type

```
double a[4];

int *p = a;

*(p+1)  === a[1]
```

        - if it int *p here, it will be move only 4 bytes
        - correct one →

```
double a[4];

double *p = a;

*(p+1)  === a[1]
```

- System call shmat() accepts a shared memory ID, shm_id, and attaches the indicated shared memory to the program's address space.
- The returned value is a pointer of type (void *) to the attached shared memory. Thus, casting is usually necessary.
- If this call is unsuccessful, the return value is -1.
- Normally, the second parameter is NULL.
- If the flag is SHM_RDONLY, this shared memory is attached as a read-only memory; otherwise, it is readable and writable.

- For the rest, we use default
  - second parameter → NULL
  - if the flag → SHM_ROONLY, if we not say anything, it will be read and write

📍 Detaching a Shated Memory Segment - shmdt()

- if we don't want to use that share memory anymore, we can use shmdt() and pass in pointer as parameter

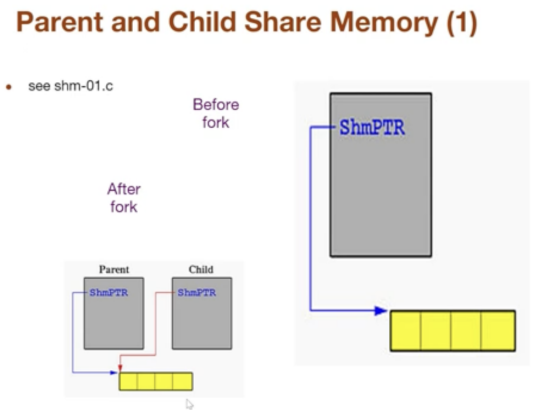
- System call shmdt() is used to detach a shared memory.
- After a shared memory is detached, it cannot be used.
- However, it is still there and can be re-attached back to a process's address space, perhaps at a different address.
- To remove a shared memory, use shmctl().
- The only argument of the call to shmdt() is the shared memory address returned by shmat().
  Thus, the following code detaches the shared memory from a program:
  - shmdt(shm_ptr);
- where shm_ptr is the pointer to the shared memory.
- This pointer is returned by shmat() when the shared memory is attached.
- If the detach operation fails, the returned function value is non-zero.

📍 Removing a Shared Memory Segment - shmctl()

- To remove a shared memory segment, use the following code:
- shmctl(shm_id, IPC_RMID, NULL);
- where shm_id is the shared memory ID. IPC_RMID indicates this is a remove operation.
- Note that after the removal of a shared memory segment, if you want to use it again, you should use shmget() followed by shmat().

    - for our class → we gonna use IPC_RMID → we'll use to remove share memory only

✅Parent and Child Share Memory



- start with basic first → share memory between parent and child
- in this case, before fork(), parent create share memory that has (fork alot ?) to store integer
- after parent fork a child, because child copy code from parent → child can access share memory wihthout calling any shmget(), ...

```
/* shm-01.c */
#include  <stdio.h>
#include  <stdlib.h>
#include  <sys/types.h>
#include  <sys/ipc.h>
#include  <sys/shm.h>
void  ClientProcess(int []);
void  main(int  argc, char *argv[])
{
    int    ShmID;
    int    *ShmPTR;
    pid_t  pid;
    int    status;
    if (argc != 5) {
        printf("Use: %s #1 #2 #3 #4\n", argv[0]);
        exit(1);
    }
```

- for this program, we have to specify four number, that number will be put to share memory by the parent, and the child then read that
- int *ShmPtr → pointer to int

```
ShmID = shmget(IPC_PRIVATE, 4*sizeof(int), IPC_CREAT | 0666);
    if (ShmID < 0) {
        printf("*** shmget error (server) ***\n");
        exit(1);
    }
    printf("Server has received a shared memory of four integers...\n");

    ShmPTR = (int *) shmat(ShmID, NULL, 0);
    if ((int) ShmPTR == -1) {
        printf("*** shmat error (server) ***\n");
        exit(1);
    }
    printf("Server has attached the shared memory...\n");
```

```
ShmPTR[0] = atoi(argv[1]);
    ShmPTR[1] = atoi(argv[2]);
    ShmPTR[2] = atoi(argv[3]);
    ShmPTR[3] = atoi(argv[4]);
    printf("Server has filled %d %d %d %d in shared
memory...\n",
          ShmPTR[0], ShmPTR[1], ShmPTR[2], ShmPTR[3]);

    printf("Server is about to fork a child process...\n");
    pid = fork();
    if (pid < 0) {
        printf("*** fork error (server) ***\n");
        exit(1);
    }
    else if (pid == 0) {
        ClientProcess(ShmPTR);
        exit(0);
    }
```

- parent put each number into each slot.
- and fork()
- and then wait
- child call clientProcess to access the data
- when child finish → parent wakeup and detach and remove share memory

```
    wait(&status);
    printf("Server has detected the completion of its child...\n");
    shmdt((void *) ShmPTR);
    printf("Server has detached its shared memory...\n");
    shmctl(ShmID, IPC_RMID, NULL);
    printf("Server has removed its shared memory...\n");
    printf("Server exits...\n");
    exit(0);
}
void  ClientProcess(int  SharedMem[])
{
    printf("   Client process started\n");
    printf("   Client found %d %d %d %d in shared memory\n",
           SharedMem[0], SharedMem[1], SharedMem[2],
SharedMem[3]);
    printf("   Client is about to exit\n");
}
```

Output

```
sarun@sarun-flowx13:~/os_course/sharemem$ ls
client  client.c  server  server.c  shm-01.c  shm-02.h
sarun@sarun-flowx13:~/os_course/sharemem$ gcc -o shm-01 shm-01.c
sarun@sarun-flowx13:~/os_course/sharemem$ ./shm-01 12 34 56 78
Server has received a shared memory of four integers...
Server has attached the shared memory...
Server has filled 12 34 56 78 in shared memory...
Server is about to fork a child process...
   Client process started
   Client found 12 34 56 78 in shared memory
   Client is about to exit
Server has detected the completion of its child...
Server has detached its shared memory...
Server has removed its shared memory...
Server exits...
sarun@sarun-flowx13:~/os_course/sharemem$
```

Child use the pointer to access shared memory

✅ Client-Server Shared Memory

(similar program above, but the processes are not parent and child)

- we gonna call the one who create share memory → server
- server create share memory and put the data in share memory
- and client read the share memory
- so, the result should be very similar to the previous one
- when 2 processes are not parent and child → what happen if the server is running and try to initialize share memory and client is run (previous one, parent finish initialize memory, then child run)
    - now, we have 2 different processes → server run and try to initialize, and client try to access the share memory, this mean we have to do some synchronization to prevent this

This can't gaurantee 100% success

```
/*shm-02.h */
#define  NOT_READY  -1
#define  FILLED    0
#define  TAKEN     1

struct Memory {
          int  status;
          int  data[4];
};
```

- we have to share int, we create this structure
  - data → we share
  - status
    - -1 → not ready
    - 0 → filled
    - 1 → taken
  - how can this status can do for syncronization? this can't be 100% garunteed

```
/*server.c*/
#include  <stdio.h>
#include  <stdlib.h>
#include  <sys/types.h>
#include  <sys/ipc.h>
#include  <sys/shm.h>
#include  "shm-02.h"
void  main(int  argc, char *argv[])
{
    key_t         ShmKEY;
    int           ShmID;
    struct Memory  *ShmPTR;
    if (argc != 5) {
        printf("Use: %s #1 #2 #3 #4\n", argv[0]);
        exit(1);
    }
```

- because the process is not parent and child, we can't use IPC_PRIVATE, we gonna use ShmKEY
- the pointer must be struct Memory → because we want to share struct

```
ShmKEY = ftok(".", 'x');
ShmID = shmget(ShmKEY, sizeof(struct Memory), IPC_CREAT | 0666);
if (ShmID < 0) {
    printf("*** shmget error (server) ***\n");
    exit(1);
}
printf("Server has received a shared memory of four integers...\n");
ShmPTR = (struct Memory *) shmat(ShmID, NULL, 0);
if ((int) ShmPTR == -1) {
    printf("*** shmat error (server) ***\n");
    exit(1);
}
printf("Server has attached the shared memory...\n");
ShmPTR->status  = NOT_READY;
ShmPTR->data[0] = atoi(argv[1]);
ShmPTR->data[1] = atoi(argv[2]);
ShmPTR->data[2] = atoi(argv[3]);
ShmPTR->data[3] = atoi(argv[4]);
```

```
    printf("Server has filled %d %d %d %d to shared memory...\n",
    ShmPTR->data[0], ShmPTR->data[1],
    ShmPTR->data[2], ShmPTR->data[3]);
    ShmPTR->status = FILLED;
    printf("Please start the client in another window...\n");

    while (ShmPTR->status != TAKEN)
        sleep(1);

    printf("Server has detected the completion of its child...\n");
    shmdt((void *) ShmPTR);
    printf("Server has detached its shared memory...\n");
    shmctl(ShmID, IPC_RMID, NULL);
    printf("Server has removed its shared memory...\n");
    printf("Server exits...\n");
    exit(0);
}
```

```
    printf("Server has filled %d %d %d %d to shared memory...\n",
    ShmPTR->data[0], ShmPTR->data[1],
    ShmPTR->data[2], ShmPTR->data[3]);
    ShmPTR->status = FILLED;
    printf("Please start the client in another window...\n");

    while (ShmPTR->status != TAKEN)
        sleep(1);

    printf("Server has detected the completion of its child...\n");
    shmdt((void *) ShmPTR);
    printf("Server has detached its shared memory...\n");
    shmctl(ShmID, IPC_RMID, NULL);
    printf("Server has removed its shared memory...\n");
    printf("Server exits...\n");
    exit(0);
}
```

- we create a key using ftok function
- and then ask for share memory in the size of struct memory (20 bytes)

- we call shmat, actually, shmat return void * (generic pointer) → but before we can use we have to specify which kind of pointer we want → cast to struct Memory *
- the way we try to snychronize
    - ShmPTR → status = NOT_READY; (server set status to not ready)

```
/* client.c */
#include  <stdio.h>
#include  <stdlib.h>
#include  <sys/types.h>
#include  <sys/ipc.h>
#include  <sys/shm.h>
#include  "shm-02.h"
void  main(void)
{
    key_t       ShmKEY;
    int         ShmID;
    struct Memory  *ShmPTR;
    ShmKEY = ftok(".", 'x');
    ShmID = shmget(ShmKEY, sizeof(struct Memory), 0666);
    if (ShmID < 0) {
        printf("*** shmget error (client) ***\n");
        exit(1);
    }
```

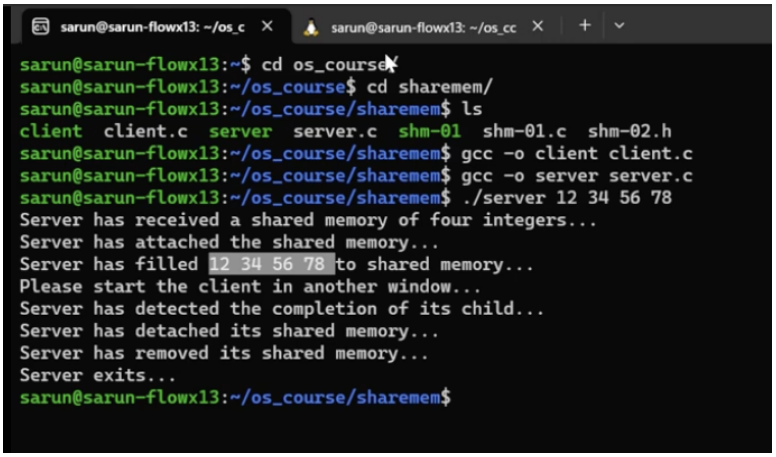
Video player

```
  printf("   Client has received a shared memory of four integers...\n");
  ShmPTR = (struct Memory *) shmat(ShmID, NULL, 0);
  if ((int) ShmPTR == -1) {
      printf("*** shmat error (client) ***\n");
      exit(1);
  }
  printf("   Client has attached the shared memory...\n");
  while (ShmPTR->status != FILLED) ;
  printf("   Client found the data is ready...\n");
  printf("   Client found %d %d %d %d in shared memory...\n",
           ShmPTR->data[0], ShmPTR->data[1],
           ShmPTR->data[2], ShmPTR->data[3]);
  ShmPTR->status = TAKEN;
  printf("   Client has informed server data have been taken...\n");
  shmdt((void *) ShmPTR);
  printf("   Client has detached its shared memory...\n");
  printf("   Client exits...\n");
  exit(0);
```

- client has to wait for status to change to FILLED.

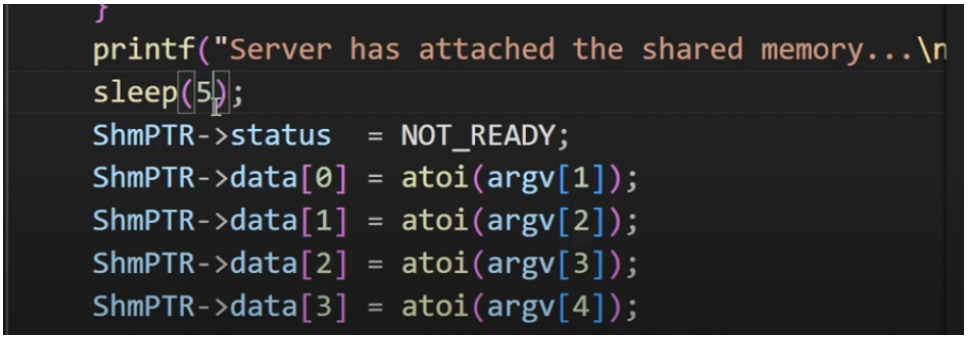
Output

to run, need to use 2 terminal

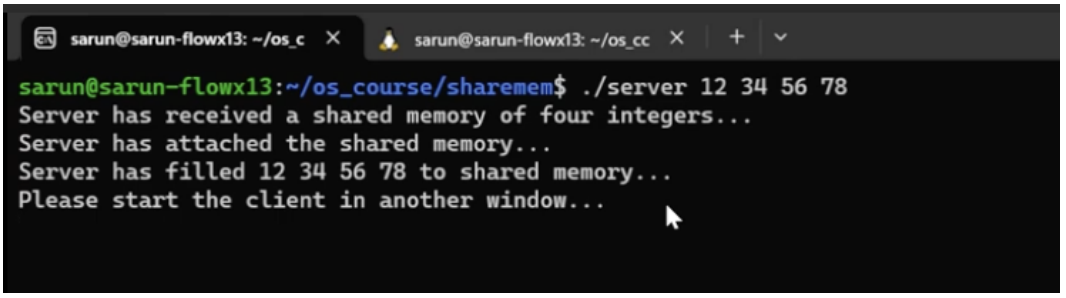


- it seems work, but at this point, we assume that the server can reach ShmPTR → status = NOT_READY before client
  - assume that server set status to be NOT_READY befroe client check the status if it failed
  - but actually, we don’t know and can’t garunteed.

Let try if server delay



Output



- server wait for client and it may wait forever, because client access share memory before we expected.
- This is why it can't garunteed

Lab7 (share memory)

2. Write a C program to do the same task as question number 1 but use share memory. The parent must remove the shared memory before existing.

The example output of the program is as follows:
./lab7_shm 5
Parent: I have created a shared memory for result...
Parent: I have attached the shared memory...
Parent: I am about to fork a child process...
Parent: Waiting for my child
Child: I am calculating
Child: The result is 15
Child: Goodbye
Parent: sum from my child is 15
Parent: I have detached the shared memory...
Parent: I have removed the shared memory...
Parent: Goodbye

```
#include  <stdio.h>
#include  <stdlib.h>
#include  <sys/types.h>
#include  <sys/ipc.h>
#include  <sys/shm.h>
#include <unistd.h>
#include <sys/wait.h>
void  childProcess(int *ShmPtr, int num);

void  main(int  argc, char *argv[])
{
     int    ShmID;
     int    *sum;
     pid_t  pid;
     int    status, num;

     if (argc != 2) {
          printf("Usage lab7_shm <number>\n");
          exit(1);
     }
     num = atoi(argv[1]);


     ShmID = shmget(IPC_PRIVATE, sizeof(int), IPC_CREAT | 066
     if (ShmID < 0) {
          printf("*** shmget error (server) ***\n");
          exit(1);
     }
     printf("Parent: I have created a shared memory for resul
     sum = (int *) shmat(ShmID, NULL, 0);
     if ( sum == NULL) {
          printf("*** shmat error (server) ***\n");
          exit(1);
     }
     printf("Parent: I have attached the shared memory...\n")

     *sum = 0; //initialize shared memory
```

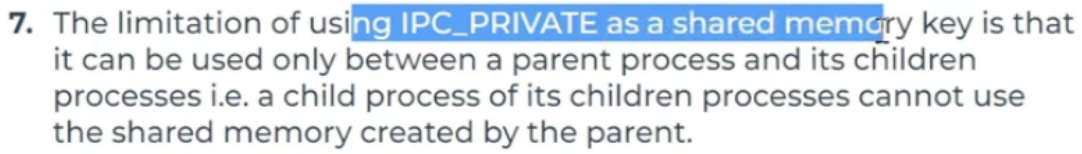
```
      printf("Parent: I am about to fork a child process...\n"
      pid = fork();
      if (pid < 0) {
           printf("*** fork error (server) ***\n");
           exit(1);
      }
      else if (pid == 0) {
           childProcess(sum, num);
           exit(0);
      }
      printf("Parent: Waiting for my child\n");
      wait(NULL);
      printf("Parent: sum from my child is %d\n", *sum);
      shmdt((void *) sum);
      printf("Parent: I have detached the shared memory...\n")
      shmctl(ShmID, IPC_RMID, NULL);
      printf("Parent: I have removed the shared memory...\n");
      printf("Parent: Goodbye\n");
      exit(0);
}

void  childProcess(int *sum, int num)
{
      printf("Child: I am calculating\n");
      int i;
      for(i =1; i <= num; i++) {
         *sum = *sum + i;
      }
      printf("Child: The result is %d\n", *sum);
      printf("Child: Goodbye\n");
}
```

Quiz

1. The system call that a process has to use to create a shared memory is
   - shmget() → create use IPC_CREAT
2. The system call that a process that needs to use the shared memory created by the process in the previous question is

- shmget()

3. The first parameter of hthe system call in the previous question is
   - shared memory key
4. The function that we can use to create the value of the parameter in the previous question is
   - ftok()
5. If a unique key is needed, _____ will be used as the first parameter of the system call (Function) that we use to create a shared memory
   - IPC_PRIVATE
6. The thing that will be returned from the function that we use to create a shared memory is _____.
   - shared memory id

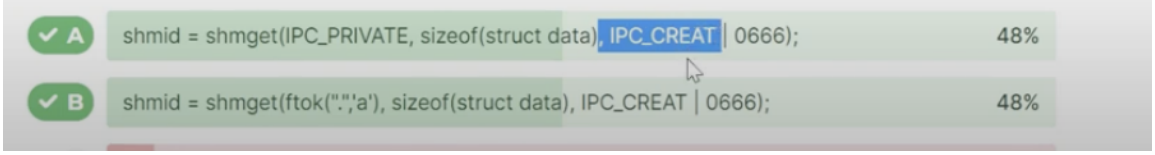


- False

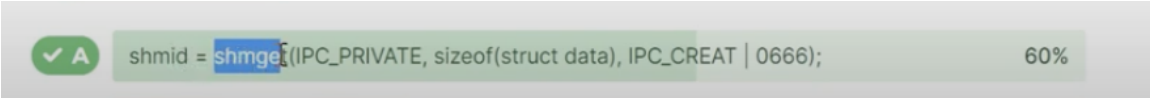
8. The second parameter of the function shmget() is the specification of _____.
   - size of the shared memory
9. If we need a shared memory that can be used to store an array of double that has 5 elements, the parameter of the shmget() fucntion will be specified as _____.
   - 5 * sizeof(double)
10. If we need to specify permission of a file as follows rwxrw-r— the octal number hta twe need to pass as paramter to the chmod command is _____.
    - 0764
11. The function that is used to assign the address of a shared memory to a variable is

- shmat()

12. The data type of the variable that is used to store the address of a shared memory must be a pointer to an integer.
    - false
13. The first parameter of the funciton shmat() is the value returned by the ___ function
    - shmget()
14. If a shared memory is created to stored data of type double, the variable to store the address of the shared memory needs to be declared as follows.
    - double * shm_ptr;
15. The function that is used to remove shared memory is
    - shmctl()
16. The parameter that will be passed to the function in the previous question to remeove a share memory is
    - IPC_RMID
17. After a parent process creates a shared memory and attached itself to the shared memory, its child process can access that shared memory.
    - True
18. Using ftok() to create a sharememory key can prevent other processes to create the same shared memory key as our process.
    - False
19. Which statement(s) can be used by a process that creates a shared memory?



20. Which statement(s) can be used by a process that creates a shared memory that will be used only be processes in the same family>



21. If a shared memory is created using the following statement;
shmid = shmget(ftok(".",'a'), sizeof(struct data), IPC_CREAT | 0666);
The variable that will be used to store the address of the shared memory needs to be declared as _____. (Choose all that apply)

- struct data *shm_ptr;

22. The statement that is used to assign the address of the shared memory to the varaubale declared in the previous question is _____.
    - shm_ptr = (struct data *) shmat(shmid, NULL, 0);
23. If we need to detach the process form a shared emmory, the statement that we will use is ____
    - shmdt(void *) shm_ptr;
24. If we need to remove a shared memory, the statemetn that we will use is _____.
    - shmctl(shmid, IPC_RMID, NULL);
    - when we remove we use shmid